

**MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA
UNIVERSITATEA DE STAT „ALECU RUSSO” DIN BĂLȚI
FACULTATEA DE ȘTIINȚE REALE, ECONOMICE ȘI ALE MEDIULUI
CATEDRA DE MATEMATICĂ ȘI INFORMATICĂ**

**DEZVOLTAREA UNUI DASHBOARD INTERACTIV
UTILIZÂND REACT ȘI RECHARTS PENTRU
ADMINISTRAREA UNUI TEREN SPORTIV**

TEZĂ DE LICENȚĂ

Autor:

Studentul grupei IS21Z

Oleg OALĂ

Conducători științifici:

Corina NEGARA

dr., conf. univ

Adela GOREA

asist. univ.

BĂLȚI, 2026

Controlată:

Data _____

Conducător științific: Corina NEGARA, dr., conf. univ

Conducător științific: Adela GOREA, asist. univ.

Aprobată

și recomandată pentru susținere

la ședința Catedrei de matematică și informatică

Procesul-verbal nr. _____ din _____

Șeful Catedrei de matematică și informatică

Vitalie ȚÎCĂU, asist. univ.

Aprobat
Şeful Catedrei de matematică şi informatică
asist. univ., Vitalie ȚÎCĂU
„____” _____ 202_

Graficul calendaristic de executare a tezei de licență

Tema tezei de licență: **Dezvoltarea unui dashboard interactiv utilizând React și Recharts pentru administrarea unui teren sportiv**, confirmată prin ordinul rectorului USARB nr.____ din „____” _____ 202_

Termenul limită de prezentare a tezei de licență la Catedra de matematică şi informatică „____” _____ 202_

Etapile executării tezei de licență:

Nr. d/o	Etapale	Termenul de realizare	Viza de executare
1.	Stabilirea temei; fixarea obiectivelor; selectarea surselor de informare		Realizat
2.	Investigația cadrului teoretic al cercetării (teoria problemei); expunerea cadrului teoretic al cercetării		Realizat
3.	Întocmirea problemei cercetării; stabilirea tipului de cercetare; elaborarea ipotezelor		Realizat
4.	Specificarea unităților (populației) studiate; construcția variabilelor (descrierea calitativă); cuantificarea (descrierea cantitativă)		Realizat
5.	Alegerea metodelor de cercetare; stabilirea tehnicilor și procedeele de lucru - în conformitate cu decizia despre caracterul lucrării: experiment de constatare, experiment formativ etc.		Realizat
6.	Culegerea datelor; selectarea modalităților de prelucrare a datelor; stocarea datelor; prelucrarea datelor; analiza datelor (verificarea ipotezelor)		Realizat
7.	Rezolvarea aspectelor de grafică și design la calculator; interpretarea rezultatelor		Realizat
8.	Formularea propunerilor de soluționare a problemei cercetării vizate în lucrare; elaborarea concluziilor și a recomandărilor practice		Realizat
9.	Susținerea preventivă a tezei		Realizat

Studentul

(semnătura)

Conducători științifici

(semnătura)

ADNOTARE

Oleg OALĂ

DEZVOLTAREA UNUI DASHBOARD INTERACTIV UTILIZÂND REACT ȘI RECHARTS PENTRU ADMINISTRAREA UNUI TEREN SPORTIV

Teză de licență. Bălți, 2026

Structura tezei:

Cuvinte-cheie:

Domeniul de studii:

Scopul cercetării:

Obiectivele cercetării:

1. Cercetarea și analiza surselor bibliografice ...
2. ...

Metodologia cercetării

Semnificația teoretică a cercetării:

Valoarea aplicativă a cercetării:

ANNOTATION

Prenume NUME

TITLE OF THESIS

Bachelor's degree. Bălți, 2026

Structure of thesis:

Keywords:

Field of study:

Research goal:

1. Research and analyze bibliographic sources ...
2. ...

The methodology of the research

The theoretical significance of the research

The practical value of the research

CUPRINS

INTRODUCERE.....	7
1. ANALIZA DOMENIULUI ȘI SPECIFICAREA CERINȚELOR	9
1.1. Descrierea domeniului.....	9
1.2. Analiza soluțiilor existente și argumentarea necesității unui sistem personalizat	10
1.3. Specificarea cerințelor funcționale și non-funcționale.....	11
2. FUNDAMENTAREA TEORETICĂ ȘI TEHNOLOGIILE UTILIZATE	13
2.1. Arhitectura sistemului client-server și utilizarea API-urilor RESTful.....	13
2.2. Tehnologii pentru interfața interactivă: Next.js, React și Recharts.....	14
2.3. Baza de date și ORM-ul: PostgreSQL prin Prisma pe Supabase	15
3. PROIECTAREA ȘI ARHITECTURA SISTEMULUI.....	16
3.1. Arhitectura aplicației și schemele de comunicare frontend-backend.....	16
3.2. Proiectarea bazei de date modelul Prisma	17
3.3. Proiectarea interfețelor utilizator	18
4. IMPLEMENTAREA PRACTICĂ A PLATFORMEI.....	21
4.1. Configurarea proiectului și structura fișierelor	21
4.2. Dezvoltarea backend-ului: API Routes în Next.js	22
4.3. Construirea interfeței: componente React și grafice Recharts	23
4.4. Module speciale: calculator bancnote și împrumuturi BLL.....	26
5. TESTAREA SISTEMULUI ȘI MANUALUL DE UTILIZARE	28
5.1. Testarea funcționalităților	28
5.2. Ghid de utilizare a platformei pentru administrator	30
CONCLUZII	32
BIBLIOGRAFIE	32
ANEXE	35
Anexa 1. Schema completă a bazei de date	35
Anexa 2. Logica de plată a datoriei.....	37
Anexa 3. Lista rutelor API	38

INTRODUCERE

Privind la modul în care s-a accelerat forțat nevoia de digitalizare, observăm că optimizarea proceselor de business a împins sistemele de management mult mai departe de rolul lor tradițional adică de simple depozite de date. Practic, dashboard-urile interactive au ajuns să constituie astăzi pilonul central al oricărei infrastructuri organizaționale care se vrea eficientă. Lucrurile stau exact așa. Am constatat, pe parcursul analizei, că valoarea reală nu rezidă în simplul acces la informație, ci în capacitatea tehnică de a izola datele relevante aproape chirurgical. Dacă ne raportăm la monitorizarea fluxurilor financiare, observăm o tranziție clară către o interfață vizuală unde utilizatorul manipulează variabilele în timp real, fără a se pierde în scripturi complexe. Aici apare avantajul competitiv. Ne lovim de o realitate în care intuiția vizuală dictează, într-o măsură mai mică sau mai mare, performanța sistemului de management.

Importanța acestor sisteme integrate este reflectată în administrarea eficientă a afacerilor sportive:

- În managementul timpului, acestea sunt utilizate pentru a monitoriza rezervările orelor de joc, evitând suprapunerile și optimizând orarul echipelor;
- În gestiunea financiară, dashboard-urile ajută la prezentarea încasărilor, a cheltuielilor și a datoriilor în mod clar și concis, facilitând evidența banilor din casă;
- În managementul administrativ, acestea sunt folosite pentru transparență, planificarea resurselor și luarea deciziilor strategice bazate pe date.

Lucrarea de față își propune dezvoltarea unui dashboard interactiv complex (aplicație full-stack) utilizând tehnologii moderne și complementare:

- Next.js 15 cu App Router framework React care gestionează simultan interfața și API
- React 19 cu hooks (useState, useEffect) pentru interfața interactivă și gestionarea stării
- Recharts biblioteca de grafice interactive și responsive integrată în proiect
- Prisma ORM + PostgreSQL prin Supabase baza de date relațională în cloud
- JWT (JSON Web Tokens) + bcryptjs pentru autentificare securizată
- Tailwind CSS pentru stilizarea rapidă și adaptabilă a interfeței
- Vercel platforma de deployment pentru producție

Prin îmbinarea acestor tehnologii, se urmărește obținerea unei aplicații web care să fie ușor de utilizat, scalabilă și performantă prin optimizarea fluxului de date.

Scopul lucrării: Cercetarea funcționalităților bibliotecii React și a librăriei Recharts și realizarea unui dashboard interactiv ce permite gestionarea eficientă a programărilor și vizualizarea în mod dinamic a datelor, ce țin de administrarea unui teren sportiv.

Pentru atingerea scopului au fost formulate următoarele obiective specifice:

- Analiza principiilor de funcționare ale framework-ului Next.js, bibliotecii React și ale librăriei Recharts;
- Analiza cerințelor de administrare ale unui stadion de minifotbal;
- Proiectarea bazei de date și a arhitecturii aplicației client-server;
- Integrarea interfeței React cu baza de date prin API Routes în Next.js;
- Testarea interfeței și a funcționalităților sistemului.

Teza de licență este structurată într-o introducere, cinci capitole, o concluzie și o bibliografie.

Primul capitol descrie problema administrării stadionului și cerințele aplicației;

Al doilea capitol oferă o fundamentare asupra arhitecturii web și a tehnologiilor utilizate;

Al treilea capitol prezintă arhitectura aplicației și structura bazei de date;

Al patrulea capitol prezintă implementarea modulelor principale ale platformei;

Al cincelea capitol acoperă testarea sistemului și manualul de utilizare.

Teza de licență este prezentată pe 38 pagini de text de bază și conține 12 figuri, o listă bibliografică din 17 surse.

1. ANALIZA DOMENIULUI ȘI SPECIFICAREA CERINȚELOR

Capitolul de față are drept scop fundamentarea necesității de elaborare a unei aplicații bazate pe web, printr-o detaliere analitică în legătură cu domeniul de referință administrarea unui teren sportiv. Un produs software pentru a fi util și viabil trebuie să rezolve câteva probleme reale din activitatea zilnică a utilizatorului. În continuare vor fi descrise procesele de gestionare legate de un stadion de minifotbal, vor fi identificate deficiențele existente în metodele tradiționale și vor fi extrase cerințele funcționale și non-funcționale necesare pentru dashboard-ul care le va îndeplini.

1.1. Descrierea domeniului

Managementul modern al unei facilități sportive, precum un stadion de minifotbal, este o activitate antreprenorială complexă care depășește simpla punere la dispoziție a unui teren de joc. În esență, acest domeniu reprezintă o intersecție constantă între gestionarea timpului, coordonarea resurselor umane angajați și clienți și controlul riguros al fluxurilor financiare. Pentru un administrator, provocarea principală nu constă în organizarea jocurilor propriu-zise, ci în menținerea unei evidențe clare a tuturor variabilelor de business care asigură rentabilitatea locației.

O analiză aprofundată a proceselor operaționale dintr-un astfel de complex sportiv evidențiază mai multe direcții critice de activitate. În primul rând, managementul rezervărilor și al timpului reprezintă nucleul afacerii. Terenul este o resursă fixă, închiriată pe intervale orare, de regulă cu ora. Evidența manuală a acestor programări, realizată frecvent prin agende clasice sau aplicații de mesagerie, este inefficientă și predispusă la erori umane. Apar frecvent situații de dublă rezervare a aceluiași interval, anulări neînregistrate la timp sau dificultăți în a identifica rapid intervalele libere pentru clienții noi.

Componenta financiară necesită o monitorizare zilnică și extrem de precisă. Activitatea stadionului generează un flux de numerar dinamic: încasări din chiria terenului și cheltuieli operaționale diverse plata utilităților, mentenanța gazonului sintetic, reparații curente ale infrastructurii, achiziția de consumabile și salarizarea personalului. Fără un sistem centralizat, calculul balanței dintre încasări și cheltuieli devine o sarcină administrativă obositoare.

O specificitate majoră a acestui domeniu, care justifică din plin necesitatea unui sistem informatic personalizat, este gestionarea echipelor și a plăților amânate. În practica comercială a stadioanelor de minifotbal, o mare parte dintre clienți sunt echipe constante care rezervă terenul săptămânal. Datorită acestei fidelități, se practică frecvent sistemul de plată periodică la finalul lunii, sau pot apărea restanțe punctuale. Evidența acestor creanțe pe suport de hârtie duce inevitabil la confuzii, pierderi financiare și chiar neînțelegeri cu clienții.

De fapt, merită subliniat că mai există o problemă specifică contextului local: împrumuturile interne ale afacerii suma pe care proprietarul o injectează din resurse proprii pentru a acoperi cheltuieli operaționale temporare. Aceste sume nu sunt cheltuieli definitive, ci cheltuieli restituibile. Fără un modul dedicat de urmărire, balanța afacerii devine practic inutilizabilă ca instrument decizional.

Metodele tradiționale de gestionare sunt limitate, fragmentate și nu oferă o perspectivă analitică. Utilizarea instrumentelor nespecializate nu permite generarea de rapoarte automate și nu oferă o interfață vizuală prin care datele să poată fi interpretate rapid. Prin urmare, automatizarea acestor procese de rutină prin intermediul unui dashboard inteligent devine nu doar un avantaj competitiv, ci o necesitate.

1.2. Analiza soluțiilor existente și argumentarea necesității unui sistem personalizat

Înainte de a pune bazele arhitecturale ale oricărui sistem informatic, ne simțim obligați să scanăm riguros soluțiile care rulează deja pe piață. Pentru gestiunea spațiilor de minifotbal, opțiunile variază de la simple unelte de uz general la platforme comerciale rigide. Dar aici apare discrepanța. O implementare de tip "off-the-shelf" nu reprezintă deloc o garanție pentru rezolvarea blocajelor specifice cu care se luptă un administrator în viața reală. Practic, am observat că aceste soluții prefabricate forțează de cele mai multe ori utilizatorul să își schimbe radical fluxul de lucru pentru a se plia pe limitele software-ului, și nu invers.

Majoritatea administratorilor de baze sportive se bazează încă pe instrumente rudimentare de tip foaie de calcul, mizând pe simplitatea Excel-ului sau a variantelor cloud din suita Google. Într-o primă fază, par soluții ideale. Însă, din perspectiva unei structuri de date relaționale, observăm rapid că acest model "plat" eșuează lamentabil în a gestiona o logică de afaceri complexă. Practic, un astfel de document nu poate impune constrângeri de integritate nimic nu împiedică sistemul să valideze două rezervări pe același teren la aceeași oră, în afara vigilenței umane.

Ne-am uitat, desigur, și spre soluțiile de tip ERP sau CRM existente pe piață. Sunt sisteme solide, n-avem ce zice. Totuși, pentru specificul administrării unui stadion, aplicarea lor este contraproductivă. Ne lovim de o redundanță tehnologică masivă, unde modulele de gestiune a stocurilor complexe sau fluxurile de marketing aglomerează inutil interfața de lucru. Mai mult, licențele costisitoare și abonamentele lunare specifice modelului SaaS nu fac decât să destabilizeze un buget de operare limitat.

Există, de asemenea, platforme web dedicate exclusiv rezervărilor pentru terenuri sportive. Deși acestea rezolvă eficient problema calendarului, prezintă un dezavantaj major din perspectiva managementului intern: lipsa unui modul financiar detaliat. Aceste aplicații nu permit o evidență granulară a banilor din casă, nu integrează cheltuielile operaționale de mentenanță și nu sunt adaptate

pentru modelul de afaceri local în care echipele pot acumula datorii lunare. Nu există niciun produs comercial la prețuri accesibile care să includă simultan: calendar cu anti-suprapunere, evidență datorii per echipă, salarizare angajați, împrumuturi restituibile și calculator de bancnote fizice.

1.3. Specificarea cerințelor funcționale și non-funcționale

În ingineria sistemelor software, etapa de specificare a cerințelor reprezintă fundamentul pe care se va construi întreaga arhitectură a aplicației. O definiție clară și riguroasă a acestor cerințe asigură că produsul final va corespunde exact nevoilor beneficiarului. Pentru dezvoltarea platformei destinate administrării stadionului de mini-fotbal, cerințele au fost extrase în urma analizei fluxurilor de lucru zilnice și au fost divizate în două categorii majore: cerințe funcționale (care descriu acțiunile pe care sistemul trebuie să le execute) și cerințe non-funcționale (care definesc atributele de calitate, performanță și securitate ale sistemului).

Cerințele funcționale dictează comportamentul aplicației. Sistemul trebuie să îndeplinească:

- Modulul de autentificare: acces exclusiv pe baza unor credențiale (utilizator + parolă criptată cu bcrypt), cu emitere de JWT la autentificare și verificare prin middleware la fiecare cerere protejată;
- Modulul de gestiune a echipelor: adăugare, editare și ștergere a profilelor echipelor de fotbal care închiriază terenul; fiecare echipă are asociată o balanță financiară (câmpul "balance" în baza de date) care reflectă datoriile curente;
- Modulul de programări (calendar): interfață de calendar cu navigare lunară, alocare de ore de joc per echipă, validare automată a suprapunerilor de interval; statusul plății poate fi "paid" (achitat imediat) sau "deferred" (amânat, generând datorie);
- Modulul financiar: introducerea sumelor încasate și a cheltuielilor clasificate pe categorii (chirie teren, salariu, electricitate, mentenanță gazon, reparații, achiziții, împrumut), cu calcul automat al balanței totale;
- Modulul de datorii: vizualizarea echipelor cu restanțe, cu posibilitatea de a înregistra plăți parțiale sau totale care actualizează simultan balanța echipei și adaugă o intrare pozitivă în modulul financiar;
- Modulul angajați: evidența personalului stadionului cu roluri, date de contact, salariu și status (activ/inactiv);
- Modulul împrumuturi BLL: urmărirea cheltuielilor restituibile sume injectate de proprietar care trebuie recuperate ulterior din încasările afacerii;
- Calculatorul de bancnote: instrument de numărare fizică a banilor din casă cu comparație automată față de soldul din sistem, pentru a detecta discrepanțe;

- Dashboard-ul analitic: carduri KPI cu banii în casă, total datorii, echipe active și rezervări de azi, plus grafice Recharts pentru încasări vs cheltuieli pe 6 luni și profit net lunar.

Cerințele non-funcționale asigură stabilitatea, performanța și calitatea experienței:

- Performanță: timpii de răspuns ai API-urilor sub 2 secunde pentru operațiunile standard;
- Responsivitate: interfața se adaptează corect pe desktop, tabletă și telefon mobil;
- Securitate: toate rutele API necesită token JWT valid; parolele sunt stocate exclusiv ca hash-uri bcrypt; cookie-ul HttpOnly previne accesul JavaScript la token;
- Portabilitate: aplicație web accesibilă din orice browser modern, fără instalare locală;
- Integritate a datelor: constrângeri referențiale din schema Prisma previn înregistrări orfane.

2. FUNDAMENTAREA TEORETICĂ ȘI TEHNOLOGIILE UTILIZATE

Odată cu stabilirea cerințelor funcționale și non-funcționale, este necesară identificarea și justificarea stivei tehnologice care va fi utilizată. Trecerea de la un simplu prototip vizual la o aplicație completă de tip full-stack presupune îmbinarea mai multor tehnologii complementare. Prezentul capitol detaliază fundamentul teoretic al arhitecturii client-server, tehnologiile utilizate pentru interfața grafică interactivă și platformele selectate pentru gestionarea logicii de business și stocarea securizată a datelor.

2.1. Arhitectura sistemului client-server și utilizarea API-urilor RESTful

Pentru a dezvolta un dashboard inteligent capabil să gestioneze fluxuri continue de date, este imperativă adoptarea unei arhitecturi software decuplate. În ingineria modernă a sistemelor web, arhitectura tradițională monolitică a fost înlocuită de modelul client-server. Această paradigmă reprezintă fundamentul pe care este construită aplicația destinată administrării stadionului.

Clientul interfața vizuală este executat direct în browserul web al utilizatorului. Rolul clientului este de a afișa interfața grafică (tabele, butoane, grafice financiare) și de a capta interacțiunile utilizatorului. Din motive de securitate și performanță, clientul nu stochează date sensibile pe termen lung și nu execută calcule financiare complexe în mod autonom.

Serverul funcționează ca nucleul central al aplicației. El este responsabil pentru logica de business: când administratorul înregistrează o plată, serverul preia informația, o validează, actualizează balanța financiară a echipei respective și înregistrează tranzacția în baza de date. Tot serverul gestionează autentificarea, asigurându-se că informațiile confidențiale nu sunt accesibile persoanelor neautorizate.

Comunicarea dintre aceste două straturi se realizează prin intermediul unui API de tip RESTful. REST utilizează protocolul HTTP pentru a facilita schimbul de date asincron. Datele sunt transmise exclusiv sub formă de obiecte JSON. Ori de câte ori interfața grafică are nevoie de date noi de exemplu, pentru a desena graficul cu încasările pe ultima lună trimite o cerere GET către un endpoint specific al serverului. La adăugarea unei noi echipe se utilizează POST, la ștergerea unui angajat se utilizează DELETE.

Particularitatea acestei aplicații față de o arhitectură clasică este că Next.js permite colocarea logicii de server direct în proiectul React, prin mecanismul numit API Routes. Fișierele din directorul `pages/api/` sunt executate exclusiv pe server nu ajung niciodată în browser. Această abordare elimină necesitatea unui server Node.js/Express separat și simplifică considerabil infrastructura de deployment.

2.2. Tehnologii pentru interfața interactivă: Next.js, React și Recharts

În arhitectura platformei dezvoltate pentru administrarea stadionului de mini-fotbal, Next.js 15 cu App Router reprezintă fundamentul proiectului. Nu este vorba despre un simplu framework de interfață. Next.js gestionează simultan randarea interfeței vizuale și execuția logicii de server. Structura proiectului utilizează o combinație hibridă: App Router pentru paginile vizuale din directorul `src/app/` și Pages Router pentru API Routes din `pages/api/`.

React 19 cu hooks este motorul interfeței vizuale. Elementul central care diferențiază React de abordările tradiționale de dezvoltare web este arhitectura sa bazată pe componente. Interfața a fost segmentată modular: meniul lateral (Sidebar), bara de navigare (Navbar), panoul principal de control, calendarele, tabelele cu echipe și angajați fiecare reprezintă o componentă React distinctă.

Pentru gestionarea stării și a apelurilor asincrone, aplicația utilizează extensiv două hooks:

- `useState` stochează temporar datele la nivel de interfață: lista echipelor, valorile din formulare, starea de loading, erorile de validare;
- `useEffect` declanșează cererile HTTP către API imediat ce o pagină este accesată.

De exemplu, în pagina de datorii, hook-ul `useEffect` pornește o cerere GET către `/api/debts` la montarea componentei. Serverul returnează un obiect JSON cu lista echipelor debitoare și suma totală. Interfața stochează răspunsul prin `useState` și randează automat tabelul interactiv.

Virtual DOM este un alt concept fundamental al React. Când administratorul înregistrează o plată și datoria unei echipe se modifică, React compară rapid versiunea nouă a interfeței cu cea anterioară și actualizează în browser exclusiv acea mică porțiune de ecran care s-a modificat. Nu se reîncarcă pagina, nu există flickering vizual.

Recharts este biblioteca de grafice aleasă pentru vizualizarea analitică a datelor financiare. Spre deosebire de alte alternative, Recharts este construită nativ pentru ecosistemul React: componentele sunt JSX pur, ceea ce face integrarea extrem de fluentă. În aplicație sunt utilizate:

- `BarChart` pentru compararea vizuală a încasărilor cu cheltuielile pe ultimele 6 luni;
- `LineChart` pentru evoluția profitului net și a încasărilor cumulate lunar;
- `ResponsiveContainer` care face graficele complet adaptabile la orice dimensiune de ecran.

Datele brute primite de la server un array de tranzacții individuale sunt agregate în interfață înainte de a fi injectate în componente. Algoritmul grupează tranzacțiile pe luni, calculează sumele per categorie și generează structura de date așteptată de Recharts.

Tailwind CSS asigură stilizarea întregii interfețe printr-un sistem de clase utilitare. Avantajul principal este viteza: nu se scrie CSS separat, iar responsivitatea se obține prin prefixe de tipul `md:` sau `lg:` direct în clasele elementelor.

2.3. Baza de date și ORM-ul: PostgreSQL prin Prisma pe Supabase

Nucleul central al persistenței informaționale este baza de date. Aplicația utilizează PostgreSQL ca sistem de gestiune a bazelor de date relaționale, găzduit în cloud prin platforma Supabase. Alegerea unui sistem relațional față de alternativele NoSQL a fost dictată de natura datelor: o plată financiară trebuie să fie legată strict de o echipă specifică, iar o rezervare trebuie să aparțină aceleiași echipe. Relațiile dintre entități sunt stricte și bine definite, ceea ce face PostgreSQL alegerea evidentă.

Supabase oferă două endpoint-uri de conexiune: unul pentru interogări tranzacționale (variabila `DATABASE_URL`) și unul direct (`DIRECT_URL`) pentru operațiunile de migrare ale schemei. Ambele sunt stocate în fișierul `.env` și nu sunt expuse în codul sursă.

Prisma ORM intermediază comunicarea dintre codul JavaScript și baza de date. În loc de a scrie interogări SQL brute, Prisma permite definirea modelelor în fișierul `schema.prisma` și generarea unui client type-safe pentru toate operațiunile. De exemplu, pentru a crea o nouă rezervare se apelează `prisma.reservation.create({ data: {...} })`, iar Prisma generează și execută instrucțiunea SQL corespunzătoare.

Un aspect critic pentru performanță în Next.js este gestionarea singleton-ului Prisma Client. Datorită mecanismului de hot-reload din mediul de dezvoltare, fără o implementare corectă s-ar crea sute de conexiuni la baza de date la fiecare salvare de fișier. Fișierul `lib/db.js` rezolvă această problemă prin stocarea instanței pe obiectul global:

```
[COD: lib/db.js Prisma Client singleton]
import { PrismaClient } from '@prisma/client';
const globalForPrisma = globalThis;
export const prisma =
  globalForPrisma.prisma ?? new PrismaClient({ log: ['error'] });
if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
export default prisma;
```

3. PROIECTAREA ȘI ARHITECTURA SISTEMULUI

Etapă de proiectare reprezintă faza de tranziție critică de la analiza cerințelor și fundamentarea teoretică la implementarea efectivă a produsului software. Acest capitol modelează arhitectura dashboard-ului, definind modul în care componentele interacționează, schemele de comunicare prin rețea și structura bazei de date.

Proiectarea este faza în care deciziile conceptuale capătă formă concretă diagrame, scheme de flux, modele de date. Nivelul de calitate al proiectării determină direct cât de ușor va fi de extins, de depanat și de menținut sistemul în lunile și anii care urmează. O proiectare precară se răzbună în producție sub formă debug-uri greu de localizat, erori de integritate și refactorizări costisitoare.

Am abordat proiectarea sistematic, pe trei direcții complementare. Mai întâi, arhitectura generală a sistemului și fluxul de date dintre straturile aplicației cum ajunge o acțiune a utilizatorului să modifice o înregistrare în baza de date din cloud. Urmează proiectarea bazei de date, care constituie fundația persistenței informaționale un model greșit de date poate compromite întreaga aplicație. În final, ne-am concentrat pe proiectarea interfeței utilizator, asigurând că structura vizuală reflectă logica operațională a administratorului, și nu invers. Toate trei direcții au fost gândite ca un sistem coerent, nu ca entități independente.

3.1. Arhitectura aplicației și schemele de comunicare frontend-backend

Sistemul a fost proiectat pe baza unui model cu trei niveluri distincte:

1. Nivelul de prezentare: interfața React randată în browser; componente vizuale construite cu Tailwind CSS și grafice generate de Recharts; navigarea între secțiuni se face prin Next.js Link, fără reîncărcarea paginii.
2. Nivelul logicii de business: fișierele din pages/api/ executate exclusiv pe serverul Next.js; validarea datelor, calculele matematice (balanță, datorii, profit net) și regulile de business (anti-suprapunere rezervări, integritate financiară).
3. Nivelul de stocare: PostgreSQL prin Prisma pe Supabase; entitățile principale și relațiile dintre ele sunt definite în schema Prisma.

Fluxul de date pentru o operațiune tipică înregistrarea unei plăți (Fig. 3.1):

1. Administratorul introduce suma în formularul de datorii și apasă "Salvează";
2. React colectează valoarea și trimite o cerere POST către /api/debts/pay, cu suma, ID-ul echipei și tokenul JWT în header;
3. Serverul validează tokenul, identifică echipa, calculează suma de plătit, actualizează câmpul balance al echipei și creează simultan o înregistrare nouă în tabelul Finance;
4. Serverul returnează Status 200 cu datele actualizate ale echipei;

5. Interfața React primește răspunsul, închide modalul și reîncarcă lista de datorii.

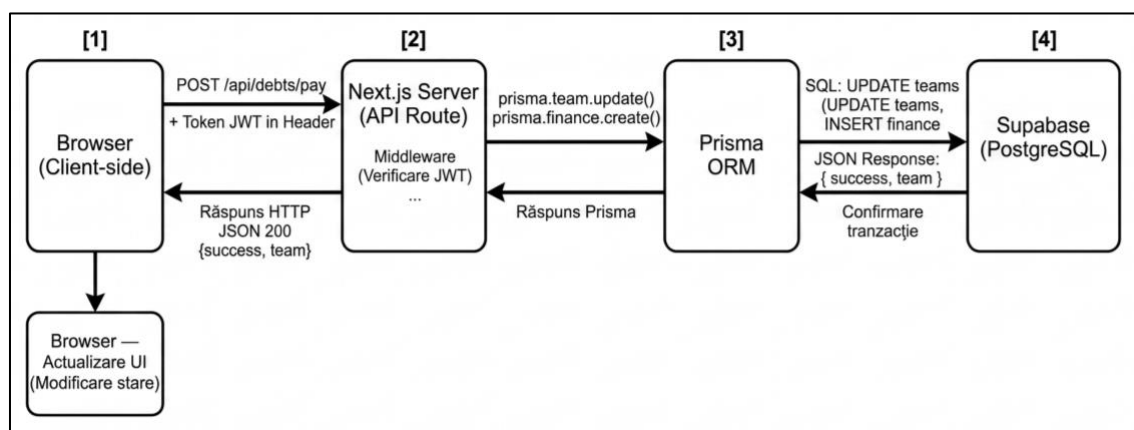


Figura 3.1. Diagrama fluxului de date cerere POST pentru plată datorie.

Schema generală de comunicare respectă principiile REST: URL-urile sunt resurse (/api/teams, /api/reservations, /api/finances, /api/debts, /api/employees, /api/loans), metodele HTTP descriu acțiunile (GET, POST, PUT, DELETE), iar răspunsurile sunt JSON cu coduri de status semantice (200, 201, 400, 401, 404, 405, 409).

3.2. Proiectarea bazei de date modelul Prisma

Proiectarea bazei de date este una dintre cele mai sensibile etape în dezvoltarea unei aplicații orientate spre gestiunea financiară. Schema a fost definită în fișierul prisma/schema.prisma și conține 7 modele principale.

Modelul User stochează datele de autentificare ale administratorilor. Câmpul password nu conține niciodată parola în text clar exclusiv hash-ul bcrypt. Câmpul username are constrângere @unique, prevenind duplicatele.

```

[COD: Fragment schema Prisma modelul User]
model User {
  id          String    @id @default(uuid())
  username    String    @unique
  password    String    // bcrypt hash
  createdAt   DateTime  @default(now())
}

```

Modelul Team reprezintă nucleul de clienți. Câmpul balance de tip Float stochează balanța financiară: valoarea 0 înseamnă că echipa nu datorează nimic, o valoare negativă (ex. -500) înseamnă o datorie de 500 lei. Aceasta este o decizie de design deliberată negativul semnalează datoria.

```

[COD: Fragment schema Prisma modelul Team]
model Team {
  id          String    @id @default(uuid())
  name        String    @unique
  phone       String?
  balance     Float      @default(0)
}

```

```

    createdAt    DateTime    @default(now())
    reservations Reservation[]
    finances      Finance[]
  }

```

Modelul Reservation gestionează programările orare. Câmpurile date, startTime și endTime sunt stocate ca String în format standardizat (YYYY-MM-DD și HH:MM). Câmpul status poate fi "paid" sau "deferred". Relația cu Team este definită prin câmpul teamId (cheie externă), iar clauza onDelete: Cascade asigură că ștergerea unei echipe elimină automat și rezervările sale.

Modelul Finance stochează fiecare tranzacție financiară. Câmpul type poate fi "income" sau "expense"; câmpul category identifică natura tranzacției. Relația opțională cu Team prin teamId permite trasabilitatea financiară per echipă.

Modelul Employee evidențiază personalul stadionului cu rolul, datele de contact, salariul și statusul activ/inactiv.

Modelul Loan urmărește cheltuielile restituibile împrumuturile proprietarului. Câmpul restituit ține evidența sumei deja returnate, iar diferența (amount - restituit) reprezintă suma rămasă de restituit.

Modelul CashCalculator este un singleton cu id fix "singleton" care persistă starea calculatorului de bancnote. Câmpurile bills și coins sunt de tip Json și stochează numărul de bancnote și monede de fiecare denominație.

3.3. Proiectarea interfețelor utilizator

Designul interfeței a urmat principii stricte de ergonomie specifice aplicațiilor de management: informația critică este vizibilă imediat după autentificare, numărul de acțiuni necesare pentru o operațiune frecventă este minimal, iar feedback-ul vizual este instantaneu.

Structura vizuală a dashboard-ului este împărțită în trei zone principale (Fig. 3.2):

1. Meniul lateral sidebar: vizibil permanent pe desktop, de tip hamburger pe mobile; conține navigarea structurată pe secțiuni stadion echipe, angajați, calendar rezervări, finanțe, datorii, calculator bancnote și grafice bare, linii, tort, geografic; afișează profilul administratorului autentificat;
2. Bara superioară navbar: conține butonul de deschidere/închidere a meniului lateral și numele utilizatorului autentificat;
3. Zona principală de conținut: aria dinamică unde Next.js randează componenta paginii selectate.

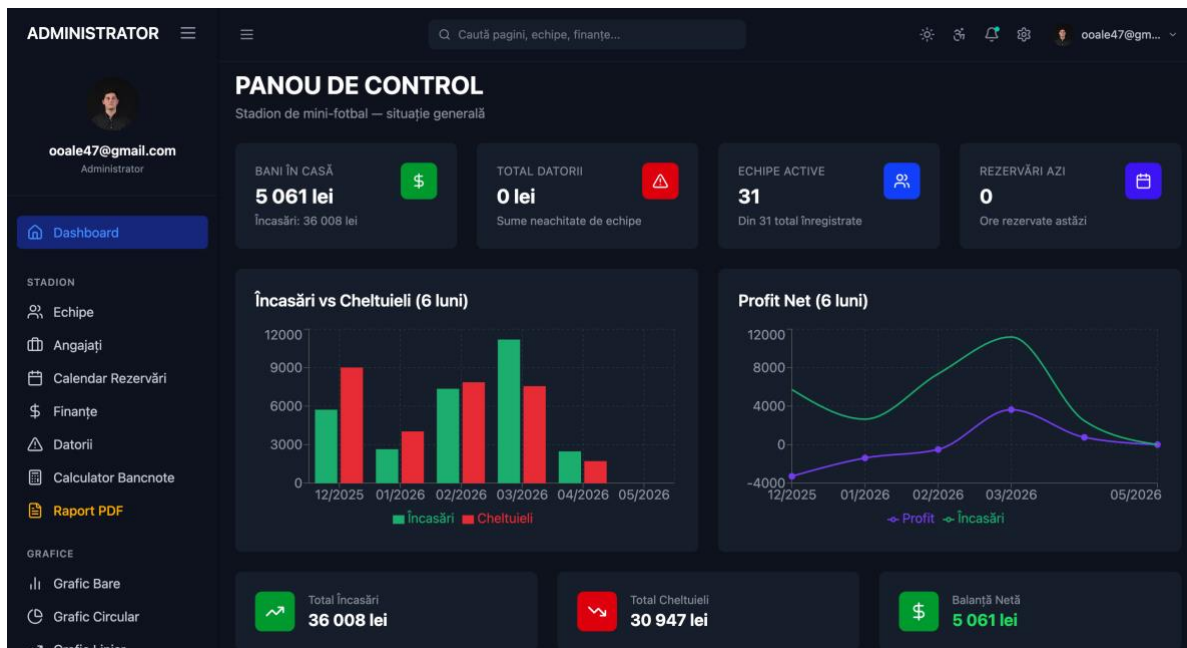


Figura 3.2. Screenshot pagina principală Dashboard carduri KPI și grafice Recharts.

Pagina principală afișează patru carduri KPI: "Bani în casă" (verde), "Total datorii" (roșu), "Echipe active" (albastru) și "Rezervări azi" (indigo). Sub acestea, două grafice Recharts: un BarChart cu încasări vs cheltuieli pe ultimele 6 luni și un LineChart cu evoluția profitului net.

Valorile sunt calculate în timp real de endpoint-ul `/api/dashboard/stats` la fiecare accesare. Pagina de calendar afișează o grilă lunară cu navigare prin săgețile prev/next. Zilele cu rezervări sunt marcate vizual (Fig. 3.3).

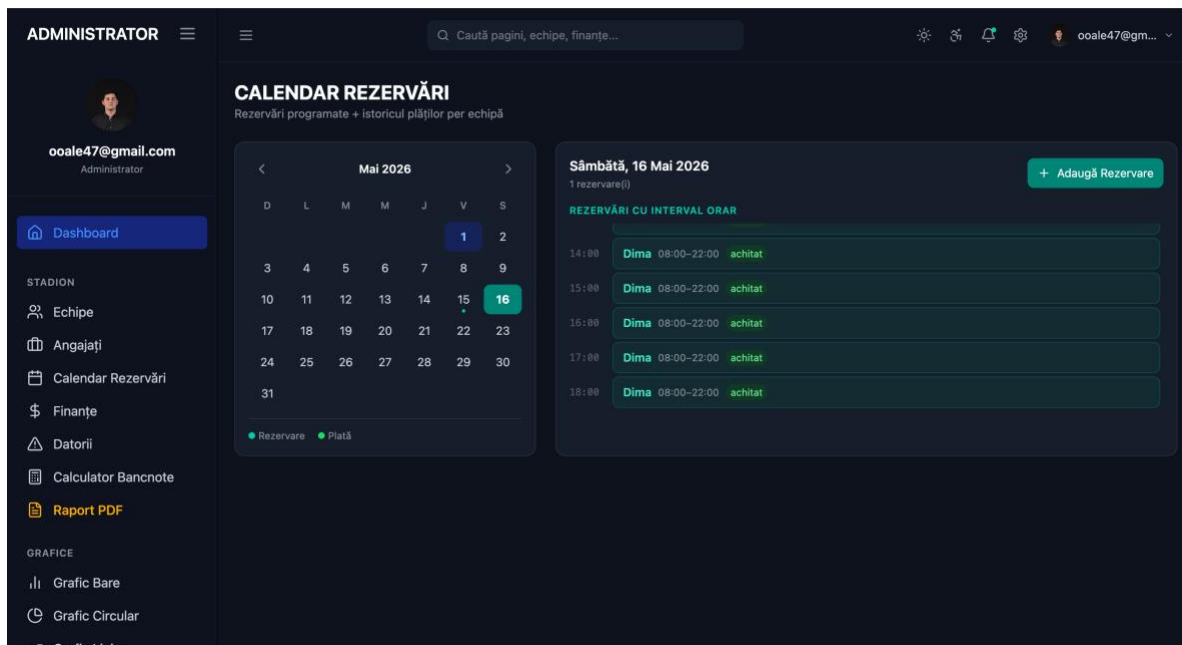


Figura 3.3. Screenshot pagina Calendar cu programările pe zi selectată.

La selectarea unei zile, panoul lateral afișează rezervările existente pentru acea zi, ordonate cronologic. Un buton "+" deschide un modal de adăugare rapidă unde administratorul introduce numele echipei, intervalul orar, costul și statusul plății (Fig. 3.4).

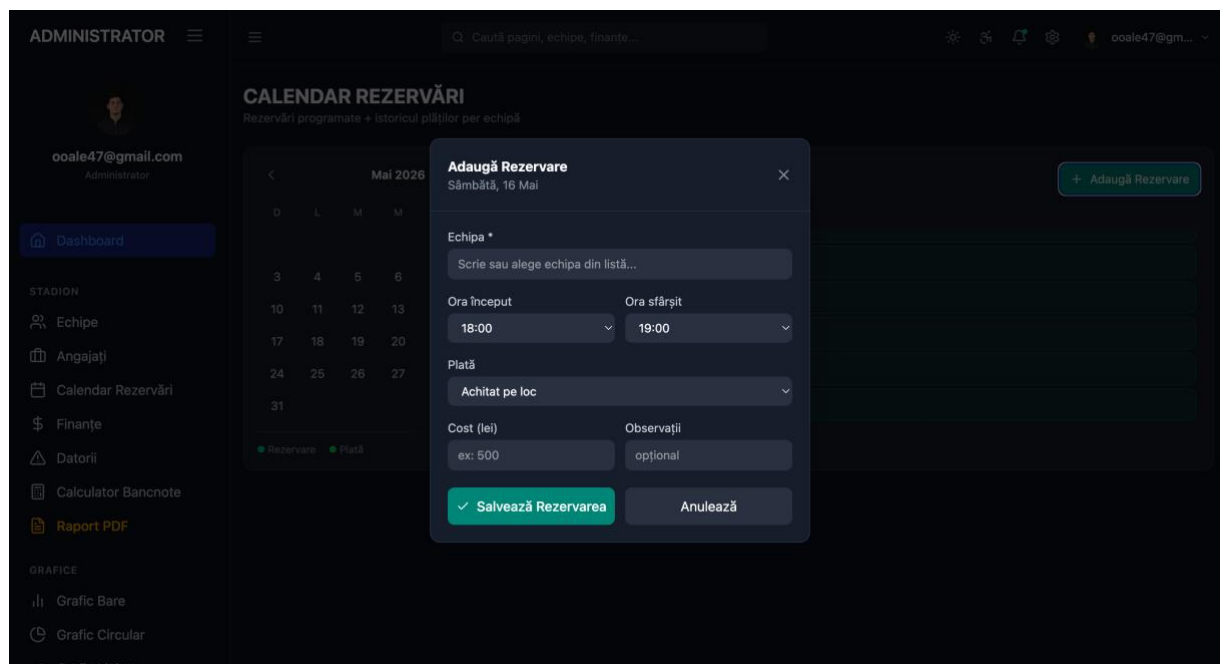


Figura 3.4. Screenshot pagina Calendar cu fereastra de programare.

O concluzie la paragraful dat sau un oaragraf de text

4. IMPLEMENTAREA PRACTICĂ A PLATFORMEI

Acest capitol descrie procesul de materializare a conceptelor teoretice și arhitecturale într-un produs software complet funcțional. Implementarea presupune scrierea efectivă a codului sursă, configurarea mediului, integrarea bibliotecilor externe și conectarea interfeței vizuale cu logica de server.

Trecerea de la proiectare la implementare este momentul în care o schemă pe hârtie devine un sistem viu un produs care răspunde la interacțiuni, persistă date și aplică reguli de business reale. Este, fără îndoială, etapa cu cel mai ridicat grad de complexitate din ciclul de dezvoltare. Ne lovim constant de discrepanțe între modelul teoretic și comportamentul efectiv al runtime-ului, de dependențe între biblioteci care se comportă diferit față de documentație sau de interacțiuni neașteptate ale mediului de execuție. Fiecare problemă de implementare rezolvată adaugă o lecție practică pe care nicio teorie nu o poate substitui.

Capitolul este organizat în patru direcții complementare: configurarea inițială a proiectului și structura fișierelor, dezvoltarea backend-ului prin API Routes Next.js, construirea componentelor React ale interfeței și, în final, descrierea modulelor cu logică specială calculatorul de bancnote și sistemul de împrumuturi BLL. Nu am tratat implementarea ca pe o activitate uniformă, pentru că nu este. Unele module, cum ar fi autentificarea sau logica de anti-suprapunere a rezervărilor, au ridicat provocări de securitate și de corectitudine care au necesitat atenție deosebită și o testare explicită, detaliată în capitolul următor.

4.1. Configurarea proiectului și structura fișierelor

Proiectul a fost inițializat cu Next.js 15 prin comanda `npx create-next-app`. Structura directoarelor reflectă arhitectura duală:

<code>pages/api/</code>	← API Routes executate pe server
<code>auth/</code>	← login, logout, register, verify
<code>dashboard/</code>	← stats (KPI + date grafice)
<code>teams/</code>	← CRUD echipe
<code>reservations/</code>	← CRUD rezervări cu logică anti-suprapunere
<code>finances/</code>	← CRUD tranzacții + calculator bancnote
<code>debts/</code>	← lista datoriei + achitare
<code>employees/</code>	← CRUD angajați
<code>loans/</code>	← CRUD împrumuturi BLL
<code>src/app/</code>	← Pagini vizuale (React Client Components)
<code>page.jsx</code>	← Dashboard principal
<code>calendar/</code>	← Calendar rezervări
<code>finances/</code>	← Modul financiar complet
<code>debts/</code>	← Evidența datoriilor
<code>employees/</code>	← Gestiunea angajaților
<code>cash-calculator/</code>	← Calculator bancnote

charts/	← Grafice dedicate (bar, line, pie, geo)
components/	← Sidebar, Navbar, AccessibilityPanel
lib/	
auth.js	← signToken, verifyToken, requireAuth
db.js	← Prisma Client singleton
prisma/	
schema.prisma	← Definiția schemei bazei de date
seed.js	← Date inițiale pentru development

Dependențele principale din package.json includ: next 15.3.1, react 19.0.0, recharts 2.15.3, @prisma/client 6.19.3, bcryptjs 3.0.3, jsonwebtoken 9.0.3, jose 6.2.3 (pentru middleware), jspdf + jspdf-autotable (pentru exportul PDF), lucide-react (iconuri UI).

4.2. Dezvoltarea backend-ului: API Routes în Next.js

Toate rutele API respectă același șablon: verificare autentificare prin `requireAuth()`, inspecție a metodei HTTP, validare a datelor de intrare, operațiune Prisma și returnare JSON. Față de un server Express separat, nu există overhead de rețea suplimentar totuși rulează în același proces Next.js.

Ruta de autentificare (`/api/auth/login`) primește username-ul și parola, caută utilizatorul în baza de date prin `prisma.user.findUnique()`, compară hash-ul `bcrypt` și, dacă credențialele sunt corecte, generează tokenul JWT și îl setează simultan ca răspuns JSON și ca cookie `HttpOnly`.

```
[COD: pages/api/auth/login.js]
export default async function handler(req, res) {
  if (req.method !== 'POST') return res.status(405).end();
  const { username, password } = req.body;
  if (!username || !password)
    return res.status(400).json({ error: 'Date lipsă' });
  const user = await prisma.user.findUnique({ where: { username } });
  if (!user || !(await bcrypt.compare(password, user.password)))
    return res.status(401).json({ error: 'Credențiale incorecte' });
  const token = signToken({ id: user.id, username: user.username });
  res.setHeader('Set-Cookie',
    `token=${token}; HttpOnly; Path=/; Max-Age=604800; SameSite=Lax`);
  res.json({ token, user: { id: user.id, username: user.username } });
}
```

Ruta rezervărilor (`/api/reservations/index.js`) implementează logica de anti-suprapunere. La primirea unei cereri POST, serverul interoghează baza de date pentru a detecta un conflict: un interval existent unde `startTime < endTime_nou AND endTime > startTime_nou`. Dacă există conflict, returnează Status 409 cu un mesaj descriptiv. Dacă terenul este liber, creează rezervarea și, în funcție de status, actualizează balanța echipei sau creează o intrare financiară.

```
[COD: Fragment pages/api/reservations/index.js verificare conflict]
const conflict = await prisma.reservation.findFirst({
  where: {
    date,
```

```

      startTime: { lt: endTime },
      endTime:   { gt: startTime }
    }
  });
  if (conflict)
    return res.status(409).json({
      error: `Interval ocupat: ${conflict.startTime}-${conflict.endTime}`
    });

```

Ruta pentru statisticile dashboard-ului `/api/dashboard/stats` execută trei interogări Prisma în paralel cu `Promise.all()`, pentru a minimiza latența. Calculează suma totală a veniturilor și a cheltuielilor, suma totală a datoriilor, numărul de rezervări de azi și generează datele lunare pentru ultimele 6 luni toate necesare pentru a alimenta graficele Recharts.

```

[COD: Fragment pages/api/dashboard/stats.js calcul statistici]
const [finances, teams, todayRes] = await Promise.all([
  prisma.finance.findMany(),
  prisma.team.findMany(),
  prisma.reservation.findMany({
    where: { date: new Date().toISOString().split('T')[0] }
  })
]);
const income = finances.filter(f => f.type === 'income')
  .reduce((s, f) => s + f.amount, 0);
const expense = finances.filter(f => f.type === 'expense')
  .reduce((s, f) => s + f.amount, 0);
const totalDebt = teams.filter(t => t.balance < 0)
  .reduce((s, t) => s + Math.abs(t.balance), 0);

```

Ruta de achitare a datoriilor (`/api/debts/pay`) implementează logica de stingere parțială sau totală. Calculează suma de plătit ca minimul dintre suma introdusă de administrator și datoria curentă a echipei prevenind înregistrarea de plăți care depășesc datoria existentă. Actualizează câmpul `balance` prin increment și creează simultan o tranzacție pozitivă în `finance`.

4.3. Construirea interfeței: componente React și grafice Recharts

Toate paginile vizuale sunt marcate cu directiva `"use client"` la prima linie, indicând Next.js că aceste componente rulează exclusiv în browser. Structura generală urmează un șablon consistent: `state` cu `useState`, `fetch` la `mount` prin `useEffect`, formulare modale pentru adăugare/editare și tabele interactive pentru listare.

Pagina de angajați exemplifică bine acest șablon. La `mount`, `useEffect` apelează `/api/employees` și stochează răspunsul în `state`. Un câmp de căutare text filtrează lista în timp real pe proprietățile `name`, `role` și `phone` (Fig. 4.1). Butonul "Adaugă Angajat" deschide un modal cu formular validat pe client (Fig. 4.2).

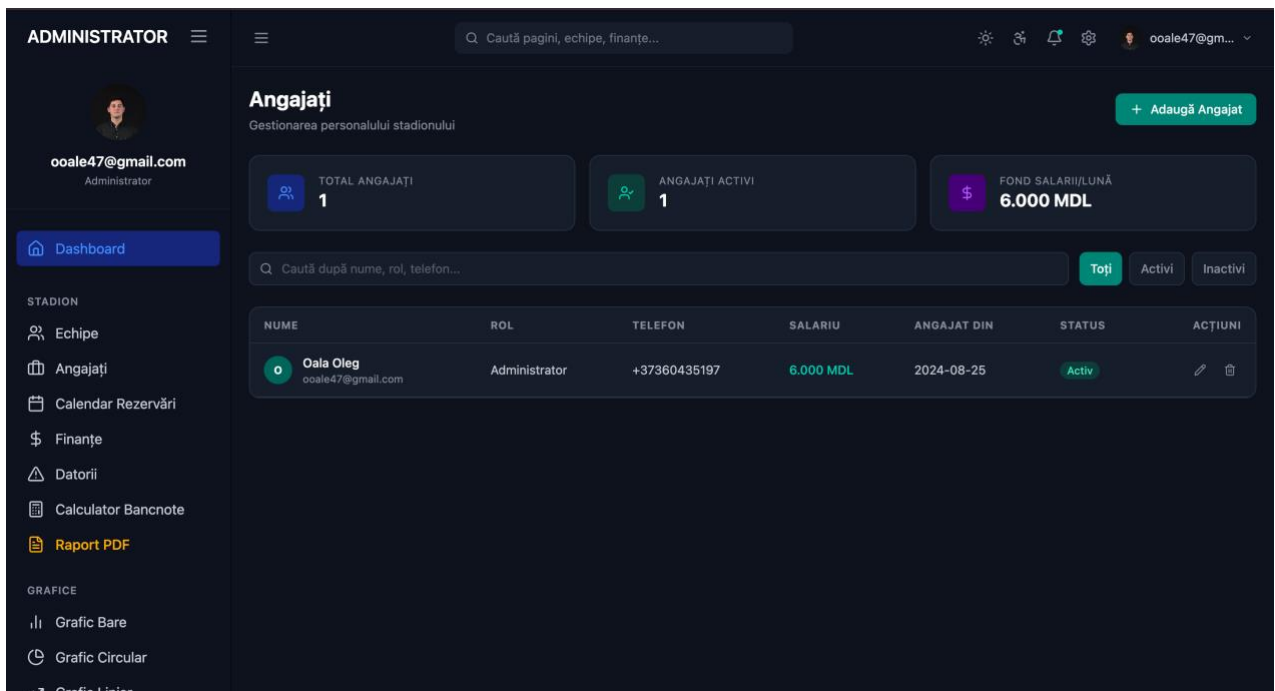


Figura 4.1. Screenshot pagina Angajați.

Figura 4.2. Screenshot cu fereastra de adăugare angajați.

La submit, componenta face POST și reîncarcă lista.

Pagina de finanțe este cea mai complexă componentă a aplicației (Fig. 4.3), (Fig. 4.4). Conține:

- tabela principală de tranzacții cu filtre avansate (tip, categorie, echipă, interval de date, interval de sume);
- funcționalitate de import CSV pentru introducerea în masă a tranzacțiilor;
- export PDF al raportului financiar prin jsPDF și jspdf-autotable;
- un subtabel separat pentru modulul de împrumuturi BLL cu operațiuni CRUD proprii.

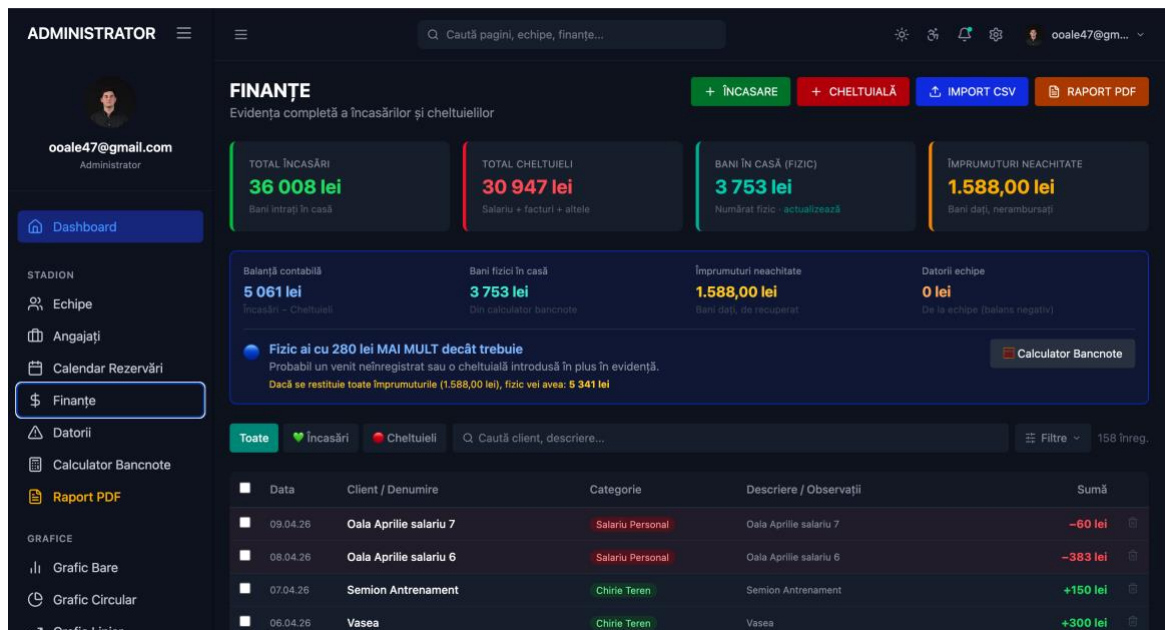


Figura 4.3. Screenshot pagina Finanțe cu filtrele avansate.

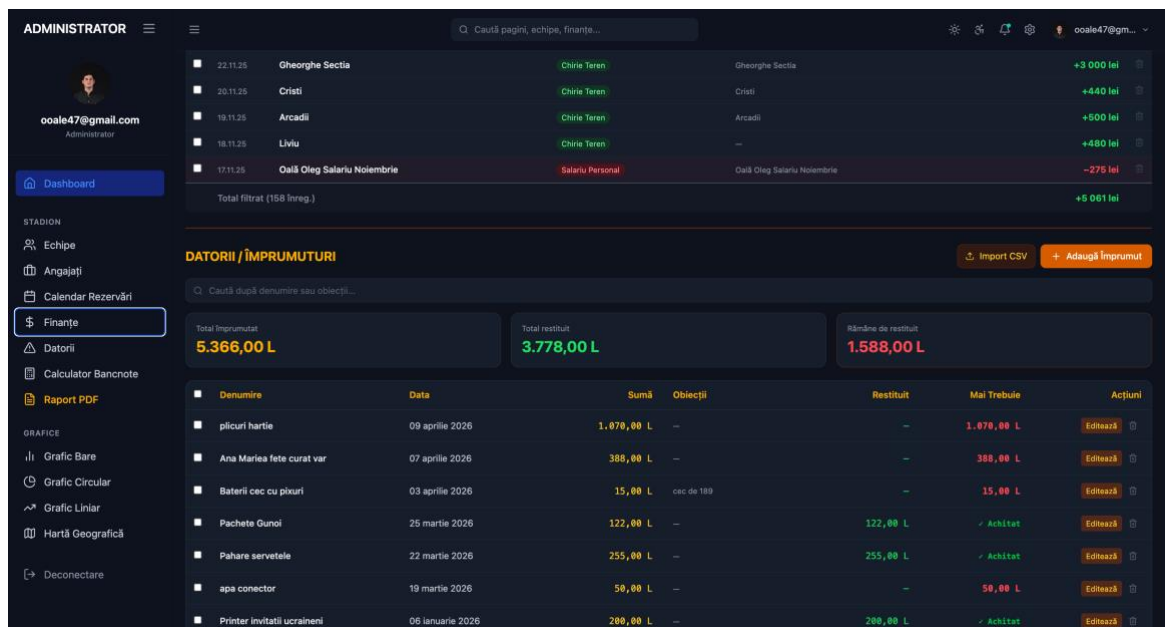


Figura 4.4. Screenshot pagina Finanțe cu subtabelul de împrumuturi.

Graficele Recharts sunt integrate în pagina principală printr-un BarChart și un LineChart (Fig. 4.5), alimentate de /api/dashboard/stats, și în secțiunea de grafice dedicate cu pagini separate .

```
[COD: Fragment grafic BarChart din pagina principală]
<ResponsiveContainer width="100%" height={250}>
  <BarChart data={stats?.monthlyData || []}>
    <CartesianGrid strokeDasharray="3 3" stroke="#374151" />
    <XAxis dataKey="month" tick={{ fill: '#9CA3AF' }} />
    <YAxis tick={{ fill: '#9CA3AF' }} />
    <Tooltip contentStyle={{
      backgroundColor: '#1F2937', border: 'none',
      borderRadius: '8px', color: '#F9FAFB'
    }} />
  </BarChart>
</ResponsiveContainer>
```

```

<Legend wrapperStyle={{ color: '#9CA3AF' }} />
<Bar dataKey="income" name="Încasări" fill="#10B981" />
<Bar dataKey="expense" name="Cheltuieli" fill="#EF4444" />
</BarChart>
</ResponsiveContainer>

```



Figura 4.5. Screenshot graficele Recharts BarChart și LineChart pe pagina principală.

Un paragraf de text

4.4. Module speciale: calculator bancnote și împrumuturi BLL

Calculatorul de bancnote este un modul fără corespondent în platformele comerciale existente, conceput specific pentru realitățile unui stadion care operează preponderent cu numerar. La finalul zilei, administratorul introduce câte bancnote și monede are în casă 200, 100, 50, 20, 10, 5, 1 lei pentru bancnote; 10, 5, 2, 1 lei pentru monede, iar aplicația calculează totalul fizic și îl compară automat cu soldul înregistrat în sistem. Diferența pozitivă sau negativă este afișată vizual cu culori verde pentru excedent, roșu pentru deficit.

Starea calculatorului este persistată în baza de date prin modelul CashCalculator singleton, astfel că la redeschiderea aplicației pe altă zi, ultimele valori introduse rămân disponibile (Fig. 4.6).

Valoare	Cantitate	Total
200 lei	3	600 lei
100 lei	3	300 lei
50 lei	36	1 800 lei
20 lei	40	800 lei
10 lei	20	200 lei
5 lei	2	10 lei
1 lei	9	9 lei
Total bancnote:		3 719 lei

Valoare	Cantitate	Total
10 lei	0	0 lei
5 lei	3	15 lei
2 lei	6	12 lei
1 lei	7	7 lei
Total monede:		34 lei

Descriere	Valoare
Total bancnote	3 719 lei
Total monede	34 lei
TOTAL FIZIC	3 753 lei

Descriere	Valoare
Trebuie să fie în casă	5 061 lei
Este la moment în casă	3 753 lei
Datorii echipe (neachitate)	0 lei
NU AJUNG — 1 308 LEI	-1 308

Valoare	Cantitate	Valoare	Cantitate
3x 200 lei	600	3x 100 lei	300
36x 50 lei	1 800	40x 20 lei	800
20x 10 lei	200	2x 5 lei	10
9x 1 lei	9	3x 5 lei	15
6x 2 lei	12	7x 1 lei	7

Figura 4.6. Screenshot Calculator Bancnote cu totaluri și comparația față de soldul sistemului.

Modulul de împrumuturi BLL rezolvă o problemă reală și frecventă: proprietarul injectează bani din resurse proprii pentru a acoperi cheltuieli operaționale urgente, cu intenția de a recupera suma ulterior. Fără un modul dedicat, aceste sume fie nu apar deloc în evidență distorsionând balanța), fie sunt înregistrate ca cheltuieli definitive inflând artificial costurile. Modulul Loan. permite înregistrarea sumei totale și actualizarea incrementală a câmpului restituit, cu afișarea sumei rămase de recuperat.

5. TESTAREA SISTEMULUI ȘI MANUALUL DE UTILIZARE

Dezvoltarea unui produs software nu se încheie odată cu scrierea ultimelor linii de cod. Validarea printr-un proces riguros de testare este obligatorie înainte de utilizarea zilnică, mai ales că dashboard-ul gestionează date financiare reale. Orice eroare de calcul sau suprapunere de rezervări poate genera pierderi de capital sau neînțelegeri cu clienții.

Testarea nu este un pas opțional adăugat la finalul procesului de dezvoltare este o activitate integrată, continuă, care însoțește fiecare etapă de implementare. Am abordat validarea sistemului pe două planuri: funcțional și de utilizare. Planul funcțional se concentrează pe corectitudinea logicii de server fiecare regulă de business critică a fost verificată prin scenarii concrete, cu valori la limită și cu date nevalide. Planul de utilizare evaluează calitatea experienței din perspectiva administratorului ușurința navigării, claritatea mesajelor de feedback și comportamentul corect al interfeței pe diferite dispozitive.

Al doilea mare subiect al acestui capitol este manualul de utilizare un ghid practic destinat administratorului care va opera platforma zilnic. Un sistem tehnic sofisticat nu are valoare dacă utilizatorul final nu îl poate folosi intuitiv și eficient. Ghidul acoperă toate fluxurile operaționale esențiale, de la autentificarea inițială până la procedura de verificare a casei la finalul zilei de lucru, structurate pas cu pas în ordinea frecvenței lor de utilizare.

5.1. Testarea funcționalităților

Testarea a fost împărțită în două direcții principale: testarea logicii de server și testarea interfeței vizuale. Testarea API-ului. S-au utilizat cereri directe prin interfața grafică a sitului și analiza răspunsurilor în terminal (Fig. 5.1).

Scenariul 1 anti-suprapunere rezervări: s-a expediat o cerere POST cu data 2025-05-15, startTime 18:00, endTime 19:00. Rezervarea a fost creată cu succes (Status 201). Imediat după, s-a expediat o a doua cerere identică. Sistemul a returnat Status 409 cu mesajul "Interval ocupat: 18:00–19:00", prevenind suprapunerea. S-a testat și un interval parțial suprapus (18:30–19:30) serverul a respins și această cerere, confirmând că logica de overlap detection funcționează corect pentru toate cazurile de suprapunere parțială.

```

GET /api/teams 304 in 811ms
GET /api/finances 200 in 1411ms
GET /api/reservations 200 in 1935ms
GET /api/teams 304 in 1861ms
GET /api/finances 304 in 1810ms
GET /api/reservations 304 in 1791ms
POST /api/reservations 201 in 1623ms
GET /api/reservations 200 in 541ms
GET /api/finances 200 in 1156ms
GET /api/teams 200 in 2056ms
POST /api/reservations 409 in 473ms
GET /api/finances 304 in 621ms
GET /api/reservations 304 in 1136ms
GET /api/teams 304 in 1893ms

```

Figura 5.1. Screenshot Terminal răspuns 409 la încercarea de rezervare suprapus.

Scenariul 2 integritatea financiară la plata datoriei: o echipă cu balance = -1000 lei. S-a trimis o cerere cu înregistrarea unei plăți = 400 lei. Serverul a incrementat balance cu 400 lei (noul balance = -600), a creat o înregistrare Finance de tip income cu amount = 400.

S-a verificat și cazul în care se încearcă plata a mai mult decât datoria existentă (amount = 1500 pentru o datorie de 800): serverul a înregistrat corect doar 800 lei, demonstrând că mecanismul Math.min() funcționează conform așteptărilor.

Scenariul 3 autentificare și protecția rutelor: acces la /api/teams fără token a returnat 401 Unauthorized. Acces cu token expirat a returnat tot 401. Acces cu token valid a returnat 200 cu lista echipelor. Middleware-ul Next.js a fost testat prin navigarea directă la /finances fără cookie redirecționare automată la /login confirmată.

Testarea interfeței s-a simulat fluxul zilnic al unui administrator:

- Validarea formularelor: introducerea de text în câmpul "Cost" din formularul de rezervare a generat mesaj de eroare vizual cu blocare a submit-ului. Câmpuri obligatorii goale au declanșat highlight roșu pe câmpurile invalide.
- Actualizarea graficelor: după înregistrarea unei noi tranzacții financiare, graficele Recharts s-au actualizat corect la navigarea înapoi la pagina principală.
- Responsivitate: aplicația a fost testată la rezoluțiile 1920×1080 (desktop), 768×1024 (tabletă) și 375×812 (telefon). (Fig. 5.2). Meniul lateral s-a ascuns corect pe rezoluții mici, tabelele au devenit scrollabile pe orizontală, cardurile KPI s-au aliniat pe verticală pe mobile.

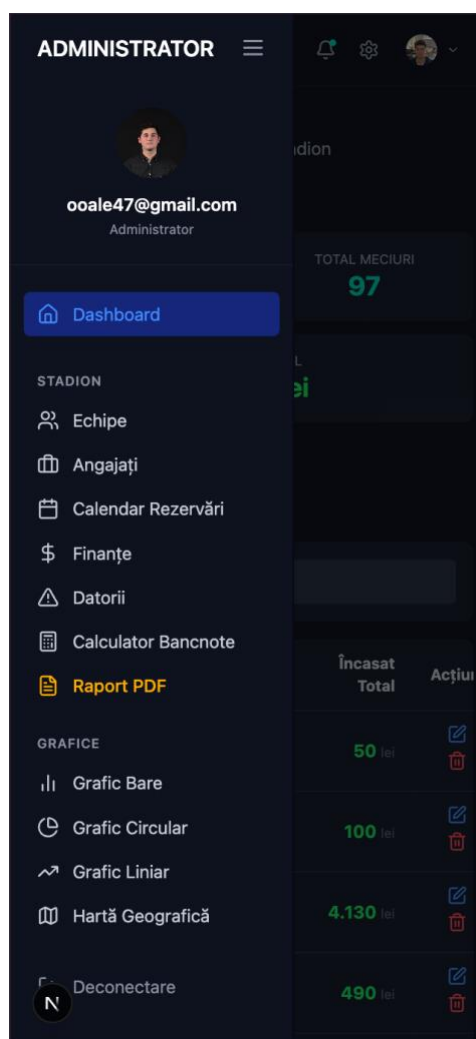


Figura 5.2. Screenshot aplicație pe mobil sidebar hamburger și carduri KPI verticale

Scenariul calculatorului de bancnote: 5 bancnote de 100 lei + 3 bancnote de 50 lei + 10 monede de 5 lei = total 700 lei. Față de soldul sistemului de 680 lei, diferența afișată a fost +20 lei excedent, marcat cu verde.

5.2. Ghid de utilizare a platformei pentru administrator

1. Autentificarea în sistem. Accesați link-ul platformei. Introduceți numele de utilizator și parola. Odată validat, veți fi redirecționat automat la panoul de control.
2. Consultarea situației financiare. Panoul central afișează instantaneu starea afacerii prin cardurile KPI (bani în casă, total datorii, echipe active, rezervări de azi) și graficele lunare.
3. Adăugarea și gestionarea echipelor. Navigați la secțiunea "Echipe". Apăsați "Adaugă Echipă Nouă", introduceți numele echipei și numărul de telefon. Balanța financiară a fiecărei echipe este actualizată automat la fiecare rezervare sau plată.

4. Efectuarea unei rezervări. Accesați "Calendar Rezervări". Faceți click pe ziua dorită, apoi pe "+". Se deschide modalul: introduceți numele echipei sau selectați din lista existentă, setați intervalul orar, costul și statusul plății. Apăsați "Salvează programarea". Dacă intervalul este deja ocupat, un mesaj roșu indică exact conflictul.
5. Gestionarea finanțelor. În "Finanțe", introduceți cheltuielile operaționale prin butonul "Adaugă Tranzacție". Filtrați tranzacțiile pe perioadă, categorie sau echipă. Exportați raportul lunar ca PDF prin butonul de export.
6. Achitarea datoriilor. În "Datorii", apăsați "Plată Nouă" în dreptul echipei respective, introduceți suma primită și data. Sistemul actualizează automat datoria echipei și adaugă suma la încasările totale.
7. Gestionarea angajaților. În "Angajați", adăugați sau editați datele personalului. Filtrați după status (activ/inactiv) sau căutați după nume.
8. Verificarea casei. La finalul zilei, accesați "Calculator Bancnote", introduceți câte bancnote și monede aveți fizic și verificați dacă totalul corespunde cu soldul din sistem. Apăsați "Salvează" pentru a păstra starea pentru a doua zi.
9. Urmărirea împrumuturilor BLL. În "Finanțe", sub tabelul principal, se află modulul de împrumuturi. Adăugați sumele injectate și actualizați câmpul "Restituit" pe măsură ce recuperați banii din încasările afacerii.

CONCLUZII

Înainte de a începe implementarea, a fost necesară o verificare a premiselor: aplicația răspunde unei nevoi reale, sau dezvoltarea este motivată exclusiv de aspectul tehnic? Răspunsul a rezultat din analiza activității unui administrator de stadion de minifotbal rezervări pe intervale orare, încasări, datorii acumulate de echipe, cheltuieli operative și salarii. Fiecare operațiune în parte este simplă, însă gestionarea lor fără instrumente adecvate produce erori cu impact financiar direct: dublă rezervare pe același interval, calcul greșit al balanței lunare sau datorii neînregistrate.

Instrumentele existente au fost analizate înainte de a stabili direcția tehnică. Foile de calcul sunt folosite frecvent în afaceri mici pentru că nu costă nimic și nu cer pregătire prealabilă, dar nu pot impune constrângeri logice. Nu există nimic care să blocheze două rezervări la aceeași oră, nimeni nu calculează automat că o echipă a intrat în datorie, iar balanța lunară se determină manual cu risc de eroare la fiecare pas. Platformele dedicate rezervărilor sportive rezolvă problema calendarului, dar nu merg mai departe niciuna nu include evidența datoriilor per echipă, un calculator de bancnote sau urmărirea împrumuturilor interne ale proprietarului.

Tehnologiile au fost alese în funcție de cerințe. Next.js 15 permite colocarea interfeței și a logicii de server în același proiect, fără două aplicații separate de întreținut. React prin hook-urile `useState` și `useEffect` a acoperit tot ce era necesar la nivel de interfață: formulare, tabele cu date actualizate în timp real și actualizări fără reîncărcarea completă a paginii. PostgreSQL accesat prin Prisma și găzduit pe Supabase a fost ales pentru că datele sunt prin natura lor relaționale o rezervare aparține unei echipe, o tranzacție poate sau nu să refere o entitate, un angajat există independent de restul. Aceste relații necesită constrângeri pe care un model bazat pe documente JSON nu le poate garanta. Securitatea a fost implementată pe două niveluri: autentificare prin JWT cu biblioteca `jose` și validare prin middleware Next.js, acoperind atât rutele API cât și navigarea în browser.

Trecerea la implementare a arătat rapid ce părți ale proiectării erau solide. Arhitectura în trei straturi interfață React în `src/app/`, logică de server în `pages/api/`, persistență prin Prisma a permis dezvoltarea fiecărui modul separat. O modificare în logica financiară nu a produs efecte asupra modului de rezervări și invers, ceea ce a simplificat atât depanarea cât și extinderea funcționalităților.

La nivelul schemei bazei de date, două decizii au redus semnificativ complexitatea codului. Stocarea balanței echipei ca valoare negativă pentru datorii elimină necesitatea unui câmp de stare separat și a verificărilor suplimentare. Directiva `onDelete: Cascade` pe relația dintre echipă și rezervările sale asigură ștergerea automată a înregistrărilor dependente fără logică suplimentară la nivel de aplicație.

Dintre cele 17 rute API implementate, două au cerut cel mai mult timp pentru a fi rezolvate corect. Condiția de detectare a suprapunerilor temporale $\text{startTime} < \text{endTime_nou} \text{ AND } \text{endTime} > \text{startTime_nou}$ prinde toate configurațiile posibile de intersecție a intervalelor, nu doar cazurile în care intervalele sunt identice. Logica de achitare a datoriei prin Math.min() între suma introdusă și soldul curent previne apariția unor balanțe pozitive fără tranzacții corespondente. Calculatorul de bancnote și modulul de împrumuturi sunt componentele cu domeniul cel mai specific din aplicație, dar în utilizarea zilnică sunt printre cele mai accesate nicio altă platformă identificată în analiza inițială nu le include pe ambele.

Testarea a confirmat comportamentul așteptat. Algoritmul de detectare a suprapunerilor returnează codul HTTP 409 inclusiv pentru suprapunerile parțiale. Toate cele trei mecanisme de securitate bcrypt, JWT și middleware returnează 401 la orice tentativă de acces neautorizat, indiferent de metoda folosită. Interfața funcționează pe dispozitive mobile: sidebar-ul devine meniu tip hamburger, tabelele permit scroll orizontal, cardurile KPI se reorganizează pe coloană. Testarea a fost manuală, pe scenarii concrete, fără un cadru automatizat. Aceasta este principala limitare a lucrării introducerea testelor unitare pentru logica de business și a testelor de integrare pentru rutele API rămâne direcția prioritară pentru dezvoltările ulterioare, alături de suportul pentru configurații multi-stadion și un modul de raportare financiară periodică.

BIBLIOGRAFIE

1. WIERUCH, R. The Road to React: Your journey to master plain yet pragmatic React.js. Independently published, 2020. ISBN 9781720043997.
2. EISENMAN, B. Learning React: A Hands-On Guide to Building Web Applications Using React and Redux. Addison-Wesley, 2017. ISBN 9780134843551.
3. LINDLEY, J. Mastering JavaScript Functional Programming. Packt Publishing, 2016. ISBN 9781785882166.
4. CHAN, E. P., BANKS, A. Learning React: Modern Patterns for Developing React Apps. O'Reilly Media, 2023. ISBN 9781098113612.
5. React Documentation React Official Site. [online] [citat 10.02.2025]. Disponibil: <https://react.dev>
6. Next.js Documentation. [online] [citat 12.02.2025]. Disponibil: <https://nextjs.org/docs>
7. Recharts Documentation. [online] [citat 15.02.2025]. Disponibil: <https://recharts.org/en-US/api>
8. Prisma ORM Documentation. [online] [citat 20.02.2025]. Disponibil: <https://www.prisma.io/docs>
9. Supabase Documentation. [online] [citat 22.02.2025]. Disponibil: <https://supabase.com/docs>
10. FreeCodeCamp React Curriculum. [online] [citat 22.02.2025]. Disponibil: <https://www.freecodecamp.org/learn>
11. MDN Web Docs JavaScript, HTML, CSS. [online] [citat 05.03.2025]. Disponibil: <https://developer.mozilla.org>
12. JWT.io JSON Web Tokens Introduction. [online] [citat 10.03.2025]. Disponibil: <https://jwt.io/introduction>
13. bcryptjs npm. [online] [citat 12.03.2025]. Disponibil: <https://www.npmjs.com/package/bcryptjs>
14. Tailwind CSS Documentation. [online] [citat 18.03.2025]. Disponibil: <https://tailwindcss.com/docs>
15. Vercel Documentation Deployment. [online] [citat 25.03.2025]. Disponibil: <https://vercel.com/docs>
16. React TypeScript Cheatsheet. [online] [citat 15.04.2025]. Disponibil: <https://react-typescript-cheatsheet.netlify.app/>
17. Frontend Masters Advanced React Patterns. [online] [citat 05.05.2025]. Disponibil: <https://frontendmasters.com/courses/advanced-react-patterns/>

ANEXE

Anexa 1. Schema completă a bazei de date

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  directUrl = env("DIRECT_URL")
}

model User {
  id          String   @id @default(uuid())
  username    String   @unique
  password    String
  createdAt   DateTime @default(now())
}

model Team {
  id          String   @id @default(uuid())
  name        String   @unique
  phone       String?
  balance     Float     @default(0)
  createdAt   DateTime @default(now())
  reservations Reservation[]
  finances    Finance[]
}

model Reservation {
  id          String   @id @default(uuid())
  teamId      String
  team        Team     @relation(fields: [teamId], references: [id], onDelete:
Cascade)
  date        String
  startTime   String
  endTime     String
  cost        Float
  status      String   @default("paid")
  notes       String?
  createdAt   DateTime @default(now())
}

model Finance {
  id          String   @id @default(uuid())
  type        String
  category    String
  client      String?
  description String?
  notes       String?
  amount      Float
  date        String
}
```

```

        teamId      String?
        team        Team?      @relation(fields: [teamId], references: [id],
onDelete: SetNull)
        createdAt   DateTime @default(now())
    }

    model Employee {
        id          String      @id @default(uuid())
        name        String
        role        String
        phone       String?
        email       String?
        salary      Float       @default(0)
        hiredAt     String
        status      String      @default("active")
        notes       String?
        createdAt   DateTime @default(now())
    }

    model Loan {
        id          String      @id @default(uuid())
        denumire    String
        date        String
        amount      Float
        notes       String?
        restituit   Float       @default(0)
        createdAt   DateTime @default(now())
    }

    model CashCalculator {
        id          String      @id @default("singleton")
        bills
@default("{\"200\":0,\"100\":0,\"50\":0,\"20\":0,\"10\":0,\"5\":0,\"1\":0}")
        coins      Json        @default("{\"10\":0,\"5\":0,\"2\":0,\"1\":0}")
        updatedAt   DateTime @updatedAt
    }

```

Json

```

export default async function handler(req, res) {
  if (!requireAuth(req, res)) return;
  if (req.method !== 'POST') return res.status(405).end();
  const { teamId, amount, date, description } = req.body;
  if (!teamId || !amount) return res.status(400).json({ error: 'Date lipsa'
});
  const team = await prisma.team.findUnique({ where: { id: teamId } });
  if (!team) return res.status(404).json({ error: 'Echipa nu exista' });
  const payAmount = Math.min(Number(amount), Math.abs(team.balance));
  await prisma.team.update({
    where: { id: teamId },
    data: { balance: { increment: payAmount } }
  });
  await prisma.finance.create({
    data: {
      type: 'income',
      category: 'achitare_datorie',
      client: team.name,
      description: description || `Achitare datorie - ${team.name}`,
      amount: payAmount,
      date: date || new Date().toISOString().split('T')[0],
      teamId
    }
  });
  const updated = await prisma.team.findUnique({ where: { id: teamId } });
  return res.json({ success: true, team: updated });
}

```

```

GET    /api/auth/verify
POST   /api/auth/login
POST   /api/auth/logout
POST   /api/auth/register

GET    /api/dashboard/stats

GET    /api/teams
POST   /api/teams
GET    /api/teams/[id]
PUT    /api/teams/[id]
DELETE /api/teams/[id]

GET    /api/reservations
POST   /api/reservations
GET    /api/reservations/[id]
PUT    /api/reservations/[id]
DELETE /api/reservations/[id]

GET    /api/finances
POST   /api/finances
POST   /api/finances/import
GET    /api/finances/cash-calculator
PUT    /api/finances/cash-calculator
DELETE /api/finances/[id]

GET    /api/debts
POST   /api/debts/pay

GET    /api/employees
POST   /api/employees
GET    /api/employees/[id]
PUT    /api/employees/[id]
DELETE /api/employees/[id]

GET    /api/loans
POST   /api/loans
GET    /api/loans/[id]
PUT    /api/loans/[id]
DELETE /api/loans/[id]
POST   /api/loans/import

```